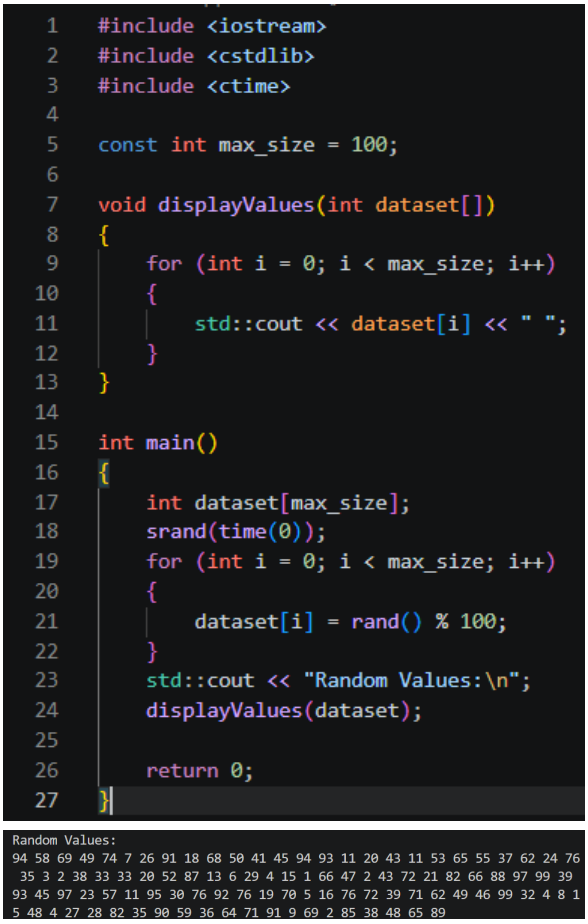
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	8/9/2026	**DATE SUBMITTED:**	10/9/2026
NAME:	Duenas Ralph Florenz F.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1

1. PROCEDURE OUTPUT

Table 7-1.	Array of Values for Sort Algorithm Testing
Code + Console Screenshot	
Observations	The code makes an array of 100 numbers and fills it with random values between 0 and 99 using rand(). It then uses a function to print all these numbers out with spaces in between them. Looking at the output, you can see a mixed-up list of 100 numbers, which shows that the random data set was made correctly.

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

const int max_size = 100;

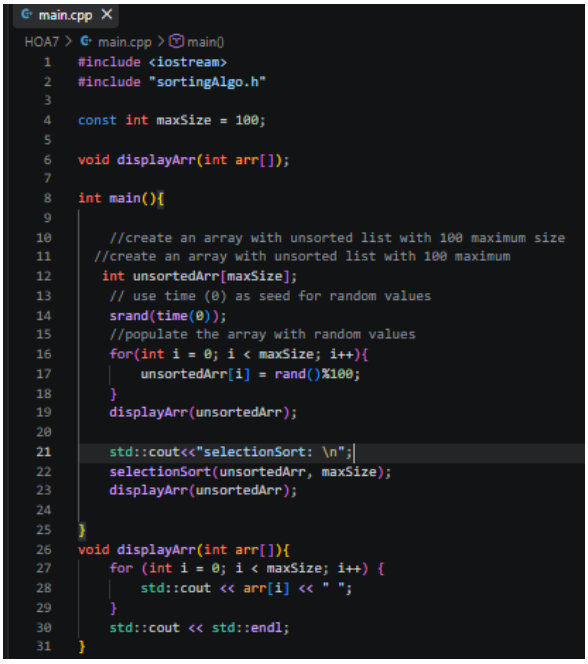
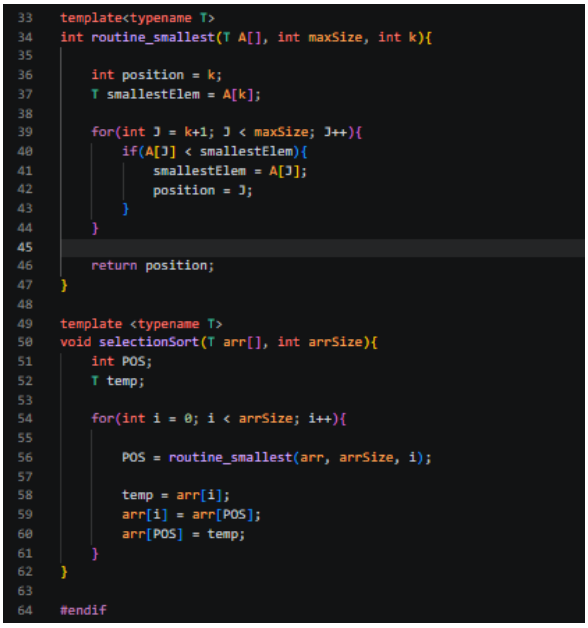
void displayValues(int dataset[])
{
    for (int i = 0; i < max_size; i++)
    {
        std::cout << dataset[i] << " ";
    }
}

int main()
{
    int dataset[max_size];
    srand(time(0));
    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand() % 100;
    }
    std::cout << "Random Values:\n";
    displayValues(dataset);

    return 0;
}
```

```
Random Values:
94 58 69 49 74 7 26 91 18 68 50 41 45 94 93 11 20 43 11 53 65 55 37 62 24 76
 35 3 2 38 33 33 20 52 87 13 6 29 4 15 1 66 47 2 43 72 21 82 66 88 97 99 39
93 45 97 23 57 11 95 30 76 92 76 19 70 5 16 76 72 39 71 62 49 46 99 32 4 8 1
5 48 4 27 28 82 35 90 59 36 64 71 91 9 69 2 85 38 48 65 89
```

Table 7-2.	Bubble Sort Technique
Code + Console Screenshot	
Observations	The code uses a bubbleSort() function saved in a header file to sort the random numbers. It uses two loops to compare neighbor numbers and swap them if the left one is bigger than the right one. The console output shows all 100 numbers lined up from lowest to highest, showing that the bubble sort worked.

Table 7-3.	Selection Sort Algorithm
Code + Console Screenshot	 

```cpp
#include <iostream>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
  //create an array with unsorted list with 100 maximum
   int unsortedArr[maxSize];
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    displayArr(unsortedArr);

    std::cout<<"selectionSort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

}
void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```cpp
template<typename T>
int routine_smallest(T A[], int maxSize, int k){

    int position = k;
    T smallestElem = A[k];

    for(int J = k+1; J < maxSize; J++){
        if(A[J] < smallestElem){
            smallestElem = A[J];
            position = J;
        }
    }

    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int POS;
    T temp;

    for(int i = 0; i < arrSize; i++){

        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}

#endif
```

	selectionSort: 1 2 4 5 5 6 6 7 8 9 9 9 11 11 12 12 12 12 14 14 15 15 16 18 18 19 19 20 20 2 3 24 25 26 27 27 27 29 30 31 32 33 37 37 38 38 38 39 41 43 45 45 46 46 47 48 48 49 50 52 52 54 54 55 59 59 60 62 63 65 66 68 68 68 70 70 70 70 73 74 74 76 78 79 79 80 82 82 82 83 84 85 86 86 87 89 90 90 90 91 93 96
Observations	This code uses a helper function to find the smallest number left in the unsorted part of the list. Then, it swaps that smallest number into its correct spot at the front. The output shows the final list fully sorted from smallest to largest, proving that finding and swapping the minimum values worked.

Table 7-4.	Insertion Sort Algorithm

Code + Console Screenshot	main.cpp
	HOA7 > main.cpp > main()
	1 #include <iostream>
	2 #include "sortingAlgo.h"
	3
	4 const int maxSize = 100;
	5
	6 void displayArr(int arr[]);
	7
	8 int main(){
	9
	10 //create an array with unsorted list with 100 maximum size
	11 //create an array with unsorted list with 100 maximum
	12 int unsortedArr[maxSize];
	13 // use time (0) as seed for random values
	14 srand(time(0));
	15 //populate the array with random values
	16 for(int i = 0; i < maxSize; i++){
	17 unsortedArr[i] = rand()%100;
	18 }
	19 displayArr(unsortedArr);
	20
	21 std::cout<<"insertionSort: \n";
	22 insertionSort(unsortedArr, maxSize);
	23 displayArr(unsortedArr);
	24
	25 }
	26 void displayArr(int arr[]){
	27 for (int i = 0; i < maxSize; i++) {
	28 std::cout << arr[i] << " ";
	29 }
	30 std::cout << std::endl;
	31 }

sortingAlgo.h

HOA7 > sortingAlgo.h > insertionSort<T>(T [], int)

```h
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

template <typename T>
void bubbleSort(T arr[], int arrSize){
    for(int i = 0; i < arrSize; i++){
        for(int j = i+1; j < arrSize; j++){
            if(arr[j] < arr[i]){
                std::swap(arr[j], arr[i]);
            }
        }
    }
}

template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1 , j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j+1]= arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}
#endif
```

```
insertionSort:
0 1 5 5 6 7 9 9 9 10 11 11 12 12 13 14 14 14 16 17 17 17 18 19 19 19 20 21 22 23 23 2
3 24 25 26 26 27 29 31 33 34 34 35 35 35 35 35 35 36 37 39 41 42 43 45 45 46 47 49 50
 51 51 53 54 54 55 55 55 57 59 60 62 63 63 64 64 64 65 65 67 70 71 76 78 79 80 84 84
87 87 88 91 92 93 95 96 98 99 99 99
```

Observations	The insertionSort() function picks up numbers one by one and compares them to the numbers on their left. It slides bigger numbers over to the right and drops the current number into its right spot. The console output shows the numbers neatly organized from 0 to 99, showing that shifting and inserting elements sorts the list properly.

SUPPLEMENTARY ACTIVITY

```cpp
#include <iostream>
#include "sortingAlgo.h"

const int maxSize = 100;

int val = 12345;

int myRand() {
    val = (1103515245 * val + 12345) % 2147483648;
    if (val < 0) {
        val = -val;
    }
    return val;
}

void displayArr(int arr[]);

int main(){

    //creates an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    //populates the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = (myRand() % 5) + 1;
    }
    displayArr(unsortedArr);

    std::cout << "Sorted Array: " << std::endl;
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    processAndCountVotes(unsortedArr, maxSize);

    return 0;
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

HOA 7 SUPP > C sortingAlgo.h > routine_smallest<T>(T [], int, int)

```h
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <string>

template <typename T>
int routine_smallest(T A[], int maxSize, int k) {
    int position = k;
    T smallestElem = A[k];

    for (int J = k + 1; J < maxSize; J++) {
        if (A[J] < smallestElem) {
            smallestElem = A[J];
            position = J;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS;
    T temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}
```

```cpp
void processAndCountVotes(int arr[], int maxSize) {
    int voteCounts[6] = {0}; // Indices 1 to 5 store votes

    for (int i = 0; i < maxSize; i++) {
        voteCounts[arr[i]]++;
    }

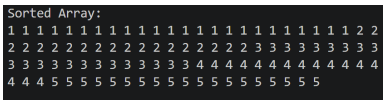
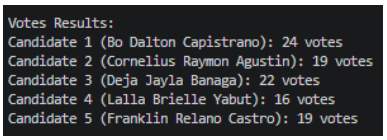
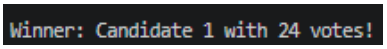
    std::string names[6] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    int winningCandidate = 1;
    int highestVotes = voteCounts[1];

    std::cout << std::endl << "Votes Results: " << std::endl;
    for (int i = 1; i <= 5; i++) {
        std::cout << "Candidate " << i << " (" << names[i] << "): " << voteCounts[i] << " votes" << std::endl;
        if (voteCounts[i] > highestVotes) {
            highestVotes = voteCounts[i];
            winningCandidate = i;
        }
    }

    std::cout << std::endl << "Winner: Candidate " << winningCandidate << " with " << highestVotes << " votes!" << st
}

#endif
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	Candidate 1 has 24 votes  Candidate 2 has 19 votes  Candidate 3 has 22 votes  Candidate 4 has 16 votes  Candidate 5 has 19 votes 	 

- **Summary of lessons learned**

In this HOA, I learned how to use and compare three simple sorting methods, which are Selection Sort, Insertion Sort, and Bubble Sort, to arrange numbers from smallest to largest. I learned that Selection Sort works by searching for the smallest number and putting it in its right spot, Insertion Sort places each item into a sorted list one at a time, and Bubble Sort works by repeatedly swapping numbers next to each other until everything is in order. I also figured out how to create my own random numbers using a math formula instead of relying on built-in code tools. Finally, I discovered how to use sorted arrays together with simple counting loops to tally up election votes correctly and find the winning candidate.

- **Analysis of the supplementary activity**

In the supplementary activity, the program shows an election voting simulation that automatically processes 100 random vote inputs for five candidates. It first initialized a dataset array in main.cpp, populated it with custom random vote IDs from 1 to 5, and displayed the unsorted results on the console. The program then passed the array into selectionSort(), which grouped identical candidate values together so processAndCountVotes() could easily tally the votes, announce each candidate's total, and declare the winner.

- **Concluding statement / Feedback:**

I think I did very well in this HOA. I was able to successfully make the code work across separate files, sort all the votes correctly, and display the right totals and winner. However, I still need more practice with writing clear template functions and figuring out how to handle a tie if two candidates get the same number of votes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concept and the flow of the code itself.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.